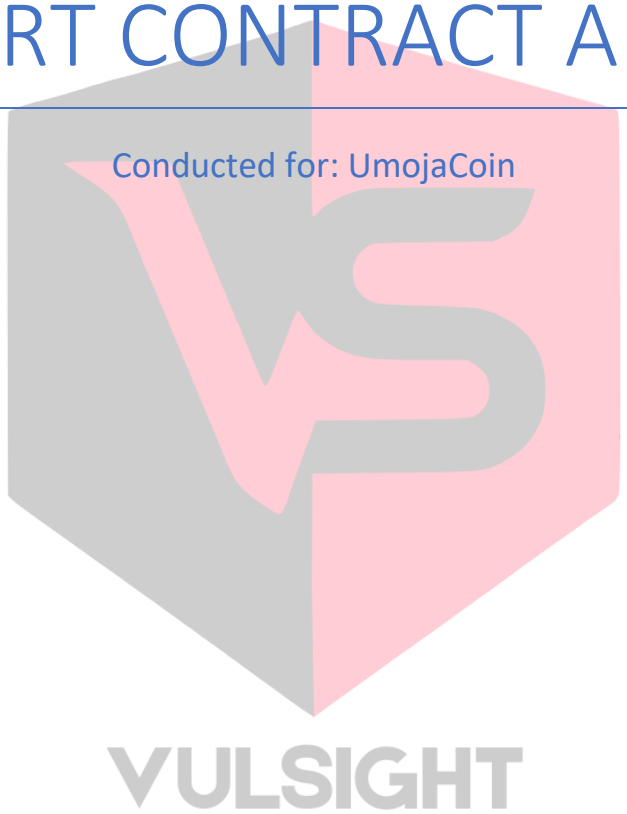
SMART CONTRACT AUDIT

Conducted for: UmojaCoin



VULSIGHT
11/12/2024

Contents

Executive Summary:

The client contacted our Consulting Firm to conduct a smart contract audit on their Solidity based smart contract. The security assessment of the UmojaCoin smart contract revealed several discrepancies between the documented specifications and the actual implementation, along with important security concerns. The most significant findings include a completely missing listing time implementation that affects the entire vesting mechanism, incorrect token distribution that deviates from the specified allocation model, and unused whitelist functionality that increases the contract's attack surface unnecessarily. While the contract inherits from well-tested OpenZeppelin contracts and implements standard security patterns, the identified issues could impact the intended token distribution and vesting mechanisms if not addressed. A low-risk finding is the missing initialization protection in the UUPS proxy pattern implementation. We recommend addressing these issues before deploying the contract to mainnet.

Project Background:

The smart contract is written in solidity. The token contract is currently deployed on the Binance Smart Chain Testnet.

Audit Scope:

Deployed Addresses	**0x3cd957699f13c5a7308fdcd244d8804bb9a64c7f**
Chain	**Binance Smart Chain Testnet**
Language	**Solidity**
Audit Completion Date	**10/12/24**

Contract Summary:

- **Contract type:** The ***UmojaCoin*** is an upgradeable ERC20 token contract that implements OpenZeppelin's upgradeable extensions for enhanced functionality and security.
- **Token standard implementation:** The contract implements the upgradeable ERC20 standard and includes UUPS (Universal Upgradeable Proxy Standard) upgradeability along with AccessControl for role-based permissions.
- **Token specifications:** The token has the symbol "UMC" (UmojaCoinToken) with a total supply of 1.5 billion tokens, distributed across different allocations.
- **Access control:** The contract uses OpenZeppelin's ***AccessControl*** pattern with an ***ADMIN_ROLE*** that can perform administrative functions like token allocation and whitelist management.

- **Vesting mechanism:** Implements a sophisticated vesting system with three categories (6-month, 12-month and 24-month).

We found:

- **0 Critical risk finding**
- **0 High risk findings**
- **3 Medium risk findings**
- **1 Low risk finding**

Severity Definitions:

Risk Level	Description
Critical	**Any kind of vulnerability that could lead to direct token or monetary loss**
High	**Any type of vulnerability that could disrupt the proper functioning of smart contract or indirectly lead to token or monetary loss.**
Medium	**Any type of vulnerability that could cause undesired actions but no serious disruption or monetary loss**
Low	**Any type of vulnerability that doesn't have any significant impact on the execution of the proper functioning of contract**

Technical Checks:

Checks	Result
Solidity version not specified	Passed
Solidity version too old	Passed
Integer overflow/underflow	Passed
Function input parameters check bypass	Passed
Insufficient randomness used	Passed
Fallback function misuse	Passed
Race Condition	Passed
Matching Project Specifications	Not Passed
Logical Vulnerabilities	Not Passed
Function visibility not explicitly declared	Passed
Use of deprecated keywords/functions	Passed
Out of Gas issue	Passed
Front Running	Passed
Insufficient Decentralization	Passed
Function input parameters lack of check	Passed
Proper function Access Control	Passed
Hardcoded Data	Passed

Code Quality and Documentation:

The audit scope included one smart contract, which was written in Solidity programming language. The code is well indented and clear in nature.

Audit Findings:

Critical findings:

Zero critical findings were found in the smart contract.

High Findings:

Zero high findings were found in the smart contract.

Medium Findings:

Three medium findings were found in the smart contract.

Missing Listing Time Implementation:

The contract lacks a critical vesting control mechanism specified in the documentation. According to the specifications, there should be a ***setListingTime*** function that allows the admin to set the starting time for all vesting schedules. This function is completely absent from the implementation.

Instead of using a configurable listing time as the reference point for vesting calculations, the contract currently uses ***block.timestamp*** directly in the ***allocateTokens*** function:

```
vestingSchedules[_beneficiary] = VestingInfo({
    amount: _amount,
    startTimestamp: block.timestamp,  // Uses block.timestamp instead of listing time
    duration: duration,
    released: 0,
    category: category
});
```

This deviation from the specifications has several implications:

1. Each allocation has its own independent start time rather than a synchronized vesting schedule
2. The admin cannot control or coordinate the start of the vesting period
3. There's no way to delay the start of vesting until token listing
4. Different users allocated tokens at different times will have desynchronized vesting schedules

Remediation:

The listing time function implementation should be added.

Unused Whitelisting Mechanism:

The contract implements a whitelist mechanism through the ***whitelisted*** mapping and ***setWhitelist*** function, but this functionality is never actually used in any operational context:

```
mapping(address => bool) public whitelisted;

function setWhitelist(address account, bool status) external onlyAdmin {
    whitelisted[account] = status;
    emit Whitelisted(account, status);
}
```

The presence of unused code:

1. Increases the contract's deployment cost unnecessarily
2. Could create confusion about the actual security model of the contract
3. May mislead integrators about available functionality

Remediation:

Either remove the unused whitelist functionality entirely or use it properly in other functions.

Token Distribution Implementation Deviates from Specifications:

The contract's token distribution implementation does not match the specified allocation requirements. According to the documentation, presale tokens should be allocated to the admin wallet, but the current implementation mints them to the treasury wallet instead. The current implementation:

```
function initialize(address admin) public initializer {
    // ... initialization code ...

    _mint(TREASURY_WALLET, EARLY_STAGE_TOKENS);
    _mint(TREASURY_WALLET, PRESALE_TOKENS);  // Incorrect: Should go to admin
    _mint(TREASURY_WALLET, COFOUNDERS_TOKENS);
    _mint(DEVELOPER_WALLET, DEVELOPERS_TOKENS);
    _mint(MARKETING_WALLET, MARKETING_TOKENS);
    _mint(TREASURY_WALLET, FREE_FOR_TRADING_TOKENS);
}
```

Specified Distribution:

1. Treasury Wallet: Early Stage, Co-founders, Free Trading tokens (1,083,700,000 tokens)
2. Admin Wallet: Presale tokens (100,000,000 tokens)
3. Developer Wallet: Developer tokens (250,000,000 tokens)
4. Marketing Wallet: Marketing tokens (66,300,000 tokens)

This mismatch results in:

1. Treasury wallet receiving excess tokens beyond specified allocation
2. Admin wallet not receiving allocated presale tokens
3. Deviation from the documented token distribution model

Remediation:
Modify the ***initialization*** function to correctly distribute tokens according to specifications.

Low findings:

One low finding was found in the smart contract.

Missing Implementation Contract Initialization Protection:

The contract inherits from ***UUPSUpgradeable*** and uses the ***OpenZeppelin initializable*** modifier, but does not implement the recommended protection against implementation contract initialization. While the initializer modifier prevents multiple initializations, the implementation contract itself remains unprotected before being used as logic for a proxy.

Although the probability of exploitation is low due to requiring a targeted approach during the deployment process, this deviates from the recommended security pattern for UUPS proxy contracts.

Remediation:
Add a constructor with ***_disableInitializers()*** to prevent implementation contract initialization.

Disclaimer:

The smart contract was tested on a best-effort basis. The smart contract was analyzed with the best security practices known at the time of writing this report. Due to the fact that the total number of test cases is unlimited and the fact that new vulnerabilities in technologies are discovered every day, the audit report makes no guarantees on the security of the contract. It is recommended to not just rely on this audit report alone. A bug bounty program for this smart contract may be created to identify any vulnerabilities within the future.